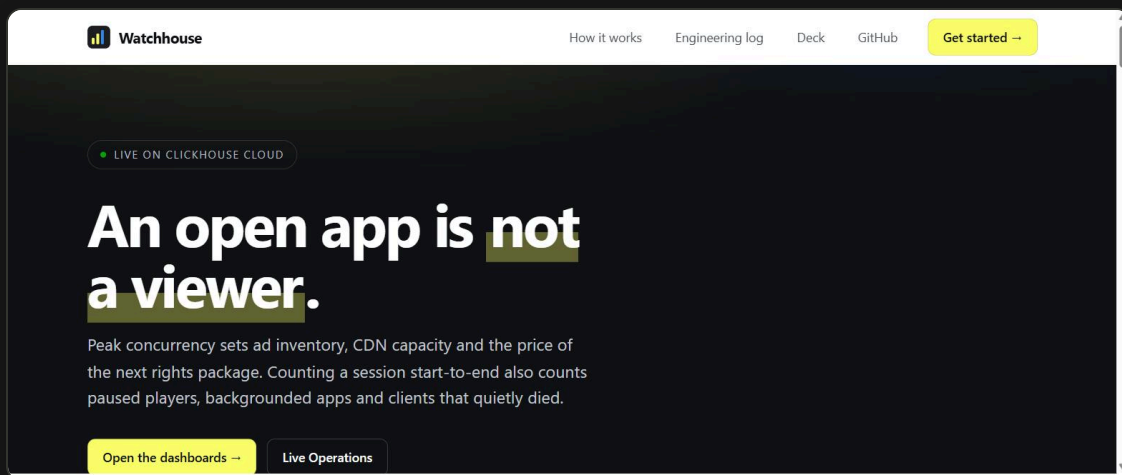




An open app is not a viewer.

Foreground-only concurrency at streaming scale, computed on ClickHouse.

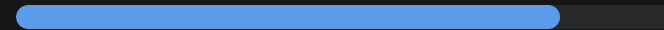


REPORTED PEAK



3,743

ACTUALLY WATCHING



3,090

653 phantom audience — 17.4% of what a session-overlap model reports

RUNNING, NOT DESCRIBED

- ClickHouse Cloud 26.2 · 905,558 rows
- ClickStack / HyperDX OTel spans on every query
- LibreChat Gemini 2.5 Flash
- Langfuse LLM trace + cost
- Kafka · Redpanda 6 partitions, keyed by session
- Redis dedup set + open sessions

Concurrency is a business number, not a vanity metric.

Peak concurrent viewers sets ad inventory, provisions CDN capacity, and justifies the price of the next rights package. It has to be defensible.

The obvious way to compute it — count a session from its first event to its last — also counts the paused player, the backgrounded app, and the client that silently went away. Nobody is watching, and the number says they are.

```
naive concurrency = interval overlap from session start → session end
```

Every figure in this deck is measured on the provided dataset: 905,558 events, 10,866 sessions, computed on ClickHouse Cloud.

653 viewers at peak who were not watching.

PEAK — NAIVE

3,743

session start → end

PEAK — FOREGROUND ONLY

3,090

active = intent $\wedge$ alive

OVER-REPORTED

17.4%

653 sessions at peak

naive session overlap

3,743

foreground-only (defensible)

3,090

On a single platform slice the gap is wider still: **ANDROID_PHONE on live content — 448 naive against 329 actual, 26.6% removed.**

Two conditions, and both are required.

```
active = intent_playing AND client_alive
```

intent_playing

Toggled only by explicit transitions. VideoPlay, AppForegrounded and resume open it; pause, AppBackgrounded and VideoSessionEnd close it.

client_alive

False during total event silence beyond 120 s — three times the measured 40 s heartbeat cadence.

We tested collapsing this into one state machine. Both variants are wrong:

- Close on a heartbeat gap → needs an explicit resume to reopen, so it **undercounts a network drop mid-playback**.
- Open on heartbeats → **overcounts every foreground pause**, and 82% of foreground pauses keep emitting heartbeats.

Measured from the data, not taken from the docs.

FACT	VALUE	WHY IT MATTERS
Heartbeat cadence (p90)	40.0 s	The data dictionary says 60 s. It is wrong. The gap threshold derives from this.
Identical across all 10 platforms	40.0 s	Device and app version do not change cadence — one threshold is justified.
Foreground pauses still emitting	82%	Why liveness alone cannot mean "watching".
Sessions where pause ≠ resume	65%	Transitions must be collapsed, never paired.
Gaps beyond 120 s	0.894%	Rare enough to be signal rather than noise.
Sessions with unstable dimensions	120	Attribute by argMin on event time, or a session double-counts.
pause / resume are not event types	—	They hide inside event_type= 'VideoHeartbeat ' . Missing this is the most common way to overcount.

Ingest, reconcile, serve — all of it in the cloud.

clients → **Kafka** (6 partitions, keyed by session) → **validator** → **dedup (Redis)** → **ClickHouse Cloud**

↓ rejected

DLQ (topic + table, reason attached)

raw_events → oracle-gated derive → **minute deltas** + **checkpoints** + **spans MV** → API → dashboards · chat

deployed: browser → **Cloud Run** (stateless dashboard) → **ClickHouse Cloud** · **GCE VM** runs ClickStack + Langfuse + LibreChat + MCP from one compose file; every query traced to HyperDX, every chat tool call traced to Langfuse

RUNNING — CLOUD RUN · GCE VM · CLICKHOUSE CLOUD

Kafka / Redpanda

Redis

DLQ

ClickHouse Cloud

Materialized views

ClickStack

Langfuse

LibreChat + MCP

DESIGNED, LABELLED AS SUCH

Schema registry upstream

Autoscaling consumers

The dashboard marks these dashed. A diagram that implies more than it runs survives one question.

THROUGHPUT MEASURED

2,028

events/s, one consumer, 6 partitions available

At-least-once delivery plus an idempotent sink.

Offsets commit only after the batch is durably in ClickHouse. A crash replays; the Redis dedup set turns the replay into a no-op. That is what "exactly-once" means in practice.

REPLAY TEST

32,815

duplicates dropped on full replay — zero double-counting

DEDUP KEY

fingerprint

session + ts + type + event — deliberately not the Kafka offset, which changes on retry

DLQ

1,703malformed events isolated with reason and payload;
pipeline never stopped

Events are keyed by `video_session_id`, so every event for a session lands on the same partition. Session reconstruction is order-dependent per key — a round-robin partitioner would make ordering unrecoverable.

The judged dataset is bigger and dirtier. We tested for that.

A pipeline that has only ever seen clean input has not been tested. So we built a deterministic fault injector — 20 fault classes across schema, event, timestamp, dimension and structural categories — and ran the loader against its output.

WHAT THE ORIGINAL LOADER DID

Bound columns **positionally**. Against a file with an extra or reordered column it does not fail — it shifts every value one column left and loads platform into country. **Wrong answers that look right.**

WHAT IT DOES NOW

Matches columns **by name** through an alias table, stages everything as String, casts in SQL where a failure is a countable zero, and **aborts loudly** if a required column cannot be resolved.

Result on a file with shuffled, renamed, widened and missing columns: 12 of 14 columns matched, 2 unrecognised ignored, 1 missing defaulted, 915 unparseable timestamps reported rather than silently zeroed.

Two rows per interval, whatever its duration.

DELTA ROWS

31,521

the model we ship

MINUTE-GRID ROWS

145,821

one row per active minute — 4.6× more

ON DISK

4.79 MiB

from a 222 MB CSV — ~46×, 5.55 bytes/row

A minute grid grows with total **watch time**. A delta model stores exactly two rows per interval regardless of duration, so it grows with the number of **intervals**. The gap widens as sessions lengthen — precisely what happens during the live events this metric exists to measure.

CODECS ARE MEASURED, NOT ASSUMED

DoubleDelta looks obviously right for 40 s heartbeats. It is **wrong here**: the sort key is `(video_session_id, event_timestamp_ms)`, so rows are ordered by session, not by time — timestamps jump at every session boundary and DoubleDelta's second difference amplifies the jump. Plain **Delta is 17.1% smaller**, at zero read cost. Three of our four codec hypotheses were wrong; the benchmark caught them. **ZSTD(6) everywhere bought 0.4% for ~20% more insert CPU** — the aggressive- compression trap. Gorilla the server refuses outright: it is a float codec.

Parity against an independent oracle — and a near-miss worth showing.

Intervals are verified against a separately written Python implementation. **35,902 intervals, exact parity.** The gate must pass before anything ships.

THE TRAP WE FELL INTO, AND CAUGHT

Writing the obvious interval-overlap query gives **2,353** instead of **3,090** — wrong by 24%, in the flattering direction. Cause: an interval that opens and closes inside the same minute contributes +1 and -1 to that minute and cancels itself. The fix is minute containment — close at the minute *after* the end minute — which is why the shipped pipeline is right and the one-line version is not.

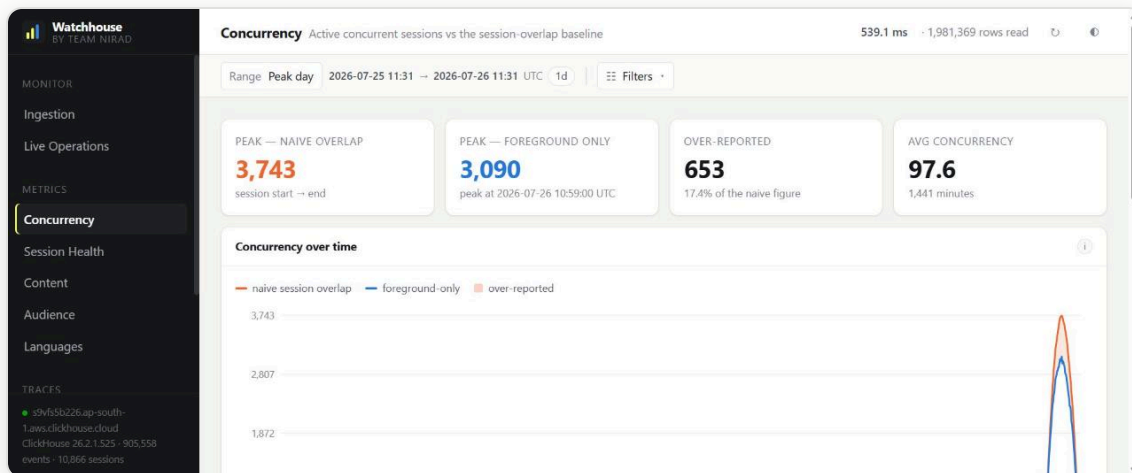
Peaks are **not additive**: platform peaks sum to 3,215 against a true peak of 3,090, because each platform peaks at a different minute. The dashboard states this rather than letting a reader add the column up.

Four ClickHouse products, each doing a real job.

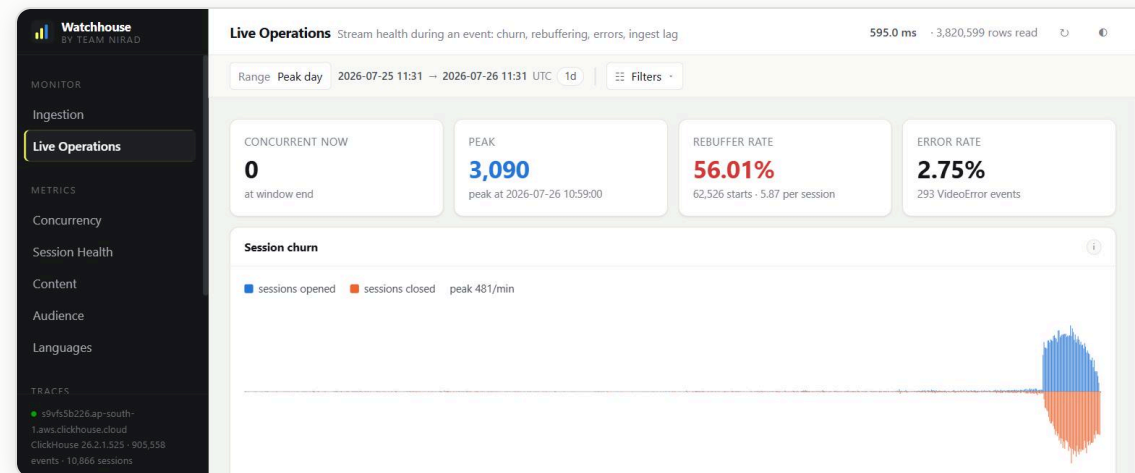
PRODUCT	WHAT IT ACTUALLY DOES HERE
ClickHouse Cloud	Landing and serving. Every dashboard figure, every playground query.
ClickStack	OpenTelemetry span on every statement, carrying duration, rows read and bytes read. The observability layer is guarded so it can never fail the query it observes.
LibreChat	Natural-language access, on Gemini 2.5 Flash.
Langfuse	Trace and cost visibility for those model calls.

Every pipeline run writes provenance to `sony.pipeline_runs` — run id, inputs, row counts, timings. **No pipeline evidence, no credit**, so the evidence is a table you can query rather than a claim in a README.

Ten views. Every number is a live query.



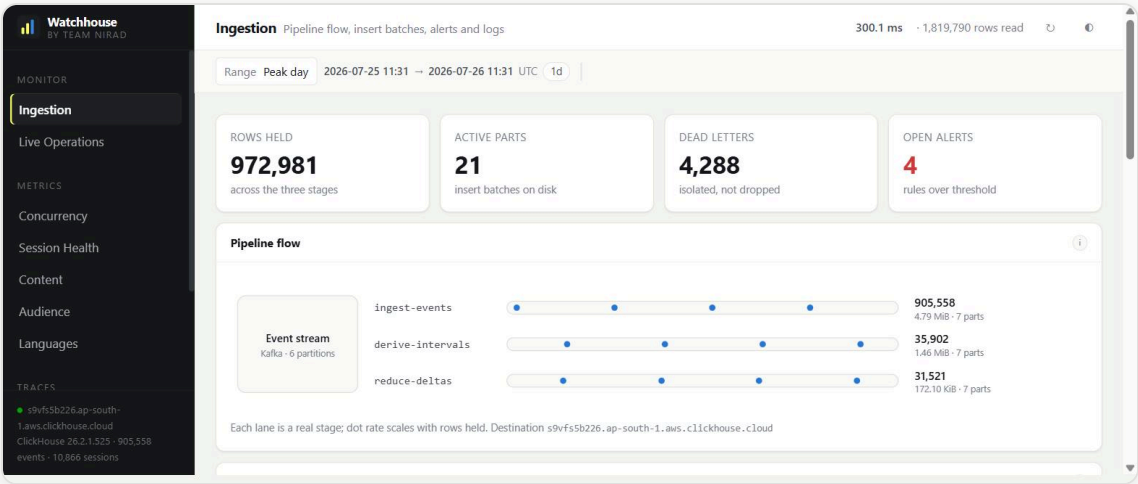
Concurrency — the gap between reported and actual, filterable across four dimensions at once. Header shows the real latency and rows read for the query that drew the chart.



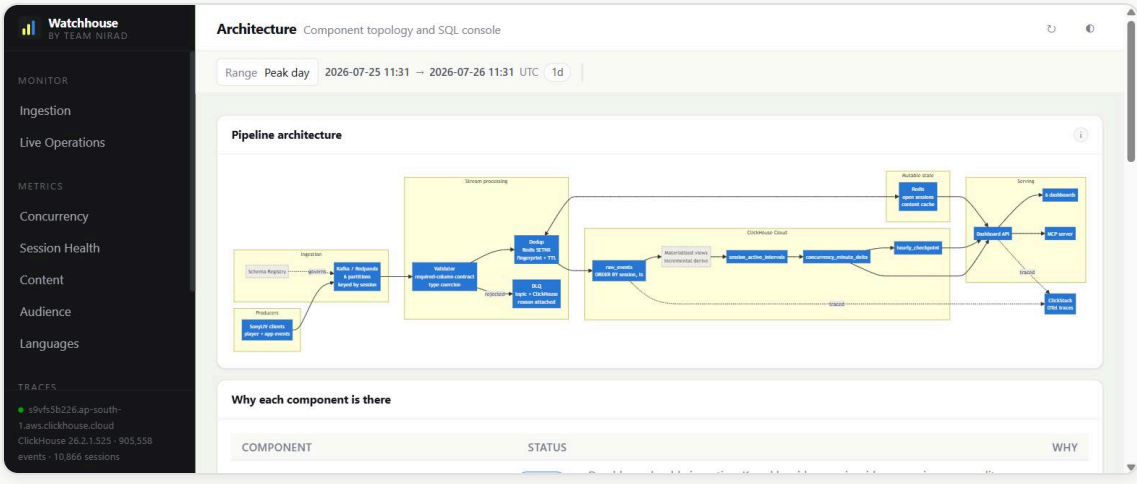
Live Operations — session churn, rebuffering and playback errors from observed QoE signals. 56% of sessions rebuffer; ANDROID_PHONE worst at 8.29 per session.

Nothing precomputed, cached or baked in: full-range chart 823 ms / 1.98M rows, peak-day slice 541 ms, per-dimension breakdown 264 ms. Vanilla SVG — no CDN, no npm, no chart library, so venue wi-fi cannot break it.

Ingestion and architecture, in the same tool.



Ingestion — three real pipeline stages as lanes, dot rate scaled from rows held. Insert batches, threshold alerts, and paginated logs with the full record behind an accordion.



Architecture — component topology with built-vs-designed marked, plus a read-only SQL console reporting p50/p95, rows and bytes read, peak memory, parts and marks.

What we found that we would rather not have.

- **Multi-device concurrency is understated.** Of 95 sessions appearing on more than one platform, **82 genuinely overlap in time** — the same viewer watching on phone and TV at once. `argMin` attributes the session to its first platform and emits one interval, so we count one stream where there are two. 0.75% of sessions; stated rather than buried.
- **Checkpoints barely pay here.** $\sim 1.01\times$ fewer rows read on this dataset, because 94% of events fall in a single day. The design wins at retention this dataset does not have. We report the measurement, not the theory.
- **The incremental path is slower than a full rebuild** on this data, for the same reason.
- **There is no geography.** Every event is `country = india`, with no state, city, region or CDN field. The Languages tab uses audio language as a proxy and says so on the card — 20.6% English sessions cannot be placed at all.

Three counting bugs were found and fixed during development, all the same species: summing values that do not sum — distinct sessions across groups, and peaks across dimensions.

Deterministic code computes. The model explains.

The MCP server exposes typed tools, not a SQL prompt. An LLM handed raw SQL over these tables writes the naive query and produces a confident, plausible, wrong number — the exact error this submission exists to eliminate. It chooses which question to ask; it never decides how concurrency is computed.

NAIVE

3,743

DEFENSIBLE

3,090

ORACLE PARITY

exact

Everything is live. Here is how to verify it yourself.

1 · THE PRODUCT

watchhouse-1045532154243.asia-south1.run.app/app

Concurrency → apply a platform filter → the curve rescope.

Architecture → the queries. Configuration → run provenance.

2 · THE CHAT FLOW

8.231.76.83:3080 — LibreChat

Login: **preethamshyam123@gmail.com**

Password: **clickhouse\$Nirad26**

Endpoint "Gemini · traced" → enable **watchhouse** MCP chip → ask "*peak concurrency on ANDROID_PHONE?*" → **6,046**.

3 · THE OBSERVABILITY

8.231.76.83:8080 — HyperDX: search service nirad-concurrency, see latency + rows read.

8.231.76.83:3000 — Langfuse (same login): tool calls, generations, token cost.

No login needed for evidence: `results/evidence/` + `results/RESULTS.md` in the repo.

Spot-check the numbers: `python scripts/verify_against_oracle.py --raw <judged.csv>` re-runs the independent oracle against whatever we claim. Demo credentials are rotated after judging.